ENSET

Département Mathématiques et Informatique
Filière(s) : BDCC II

Architecture micro-services, IA et développement full-stack

Réalisé par :

OUAZZANI TAYBI Ilyas

Supervisé par :

YOUSSEFI Mohammed

Rapport technique - Épreuve ENSET / ADRIA 2027

Parties A à F, H et I

Année Universitaire : 2026/2027

Introduction et démarche

L’objectif est de réaliser une plateforme bancaire permettant de consulter les comptes, enregistrer des transactions et effectuer des virements. J’ai organisé la solution autour de deux microservices métier indépendants, complétés par la découverte de services, la configuration centralisée, une gateway et un assistant IA. Kafka diffuse les événements de transactions après leur persistance.

Ce rapport répond aux questions A à F de l’énoncé. Pour chaque partie, il présente la réponse apportée, la méthode de réalisation et les résultats des vérifications. Les captures de tests sont regroupées en annexe. La partie G (client Angular ou Flutter) n’a pas été réalisée. Les parties H et I complètent le travail présenté.

Plan du rapport

A. Préparation et organisation du projet
B. Diagramme de classes et architecture
C. Comptes, transactions et tolérance aux pannes
D. Gateway, Eureka et Config Server
E. Assistant IA et test REST
F. Publication et consommation Kafka
H. Détection de fraude
I. Docker et intégration continue
Validation et captures des tests

Architecture micro-services, IA et développement full-stack

ENSET / ADRIA - Rapport des parties A à F, H et I

30 octobre 2026 - Démonstration bancaire. Présentation de la réalisation et des tests de la plateforme bancaire.

Dépôt : https://github.com/ilyasouazzani/OUAZZANI_TAYBI_Ilyas_test_adria_2027

Périmètre prioritaire : A à F, selon le périmètre retenu. La partie G est hors périmètre. H est exécutée ; I fournit les fichiers de déploiement, sans exécution complète de la stack Docker. Gemini remplace OpenAI dans cette réalisation.

Vérification	Résultat
Configuration centralisée	PASS
Découverte Eureka	PASS
GraphQL comptes	PASS
GraphQL transactions	PASS
Gateway	PASS
Règles métier	PASS
Kafka Docker	PASS
Outils MCP de l’agent	PASS
Réponse Gemini avec solde réel	PASS

Build validé : 31 tests automatisés sans échec, parent Maven et six modules compilés. Une validation HTTP séparée vérifie les applications ensemble. Un résultat manquant ou indisponible n’est pas remplacé par une preuve fabriquée.

A - Préparation et organisation

Question de l’énoncé : Créer un projet Maven multimodule, versionner le code et livrer un rapport PDF.

J’ai créé un pom parent pour partager les versions des dépendances, puis un module exécutable par service. Cette organisation permet de compiler toute la plateforme avec Maven tout en lançant les applications séparément. Le .gitignore exclut les secrets et les répertoires target.

Le parent Maven centralise Java 17, Spring Boot 3.5.7, Spring Cloud 2025.0.0 et Spring AI 1.1.8. Il agrège account-service, transaction-service, discovery-service, config-service, gateway-service et ai-service.

Les services métier suivent les couches domain, repository, service, web/rest, web/graphql et mcp. Chaque service conserve sa propre base H2. Le fichier .env est exclu de Git ; les valeurs réelles des clés ne figurent pas dans le rapport.

Dans cette réalisation, l’agent utilise GEMINI_API_KEY à la place de la configuration OpenAI initiale. La création ou la révocation des clés s’effectue dans le compte du fournisseur ; une clé communiquée dans un chat doit être remplacée.

Le code est versionné dans le dépôt GitHub du candidat. Les secrets et les fichiers de compilation sont exclus des livrables versionnés.

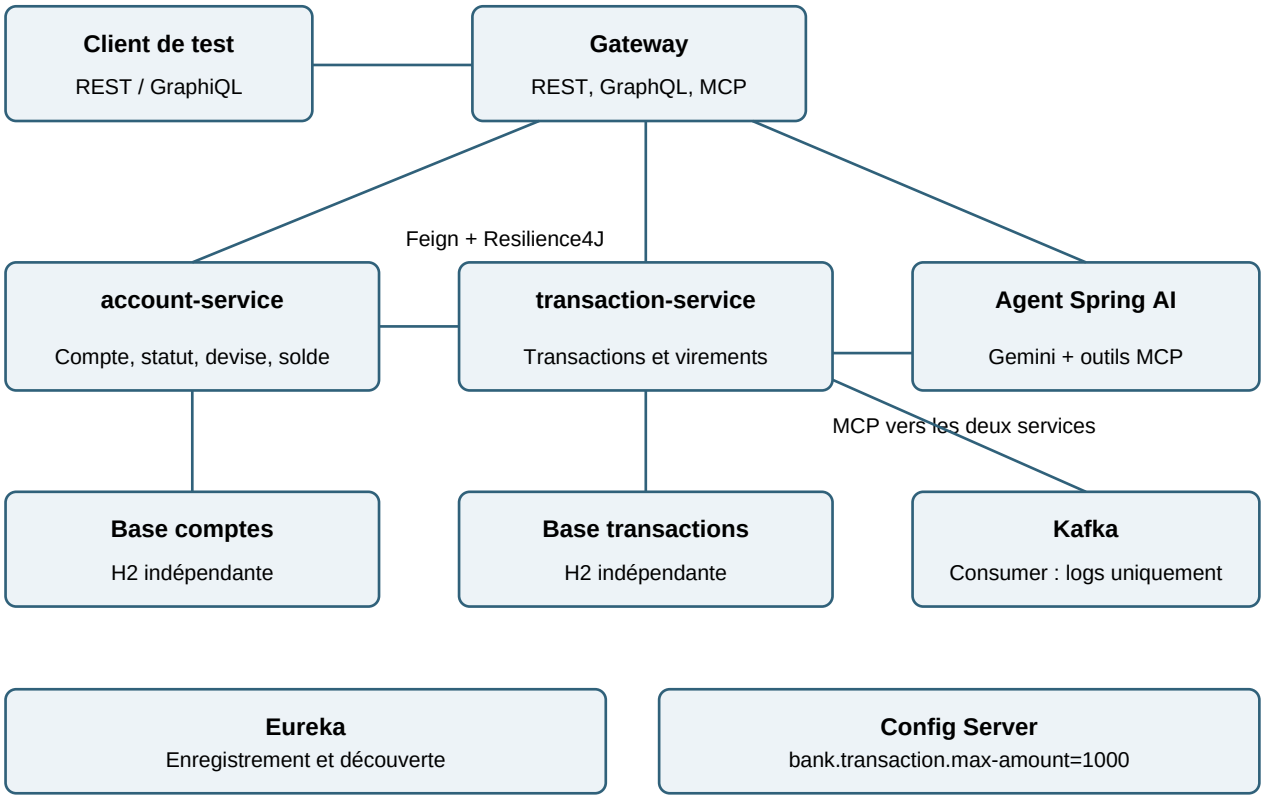
Les démarches Google Sheet, visibilité finale du dépôt et dépôt Classroom doivent être vérifiées par le candidat ; aucune soumission Classroom n’a été exécutée par l’assistant.

B - Conception et architecture

Question de l'énoncé : Établir un diagramme de classes et un schéma d'architecture.

J'ai séparé les données de compte des données de transaction. Le schéma ci-dessous situe les composants techniques et les échanges synchrones et asynchrones. Le diagramme de classes précise ensuite les attributs des deux entités et leurs références par UUID.

Deux services métier, des bases séparées et une communication par contrats réseau. Le diagramme de classes Mermaid et le schéma d'architecture Mermaid sont livrés dans docs/.

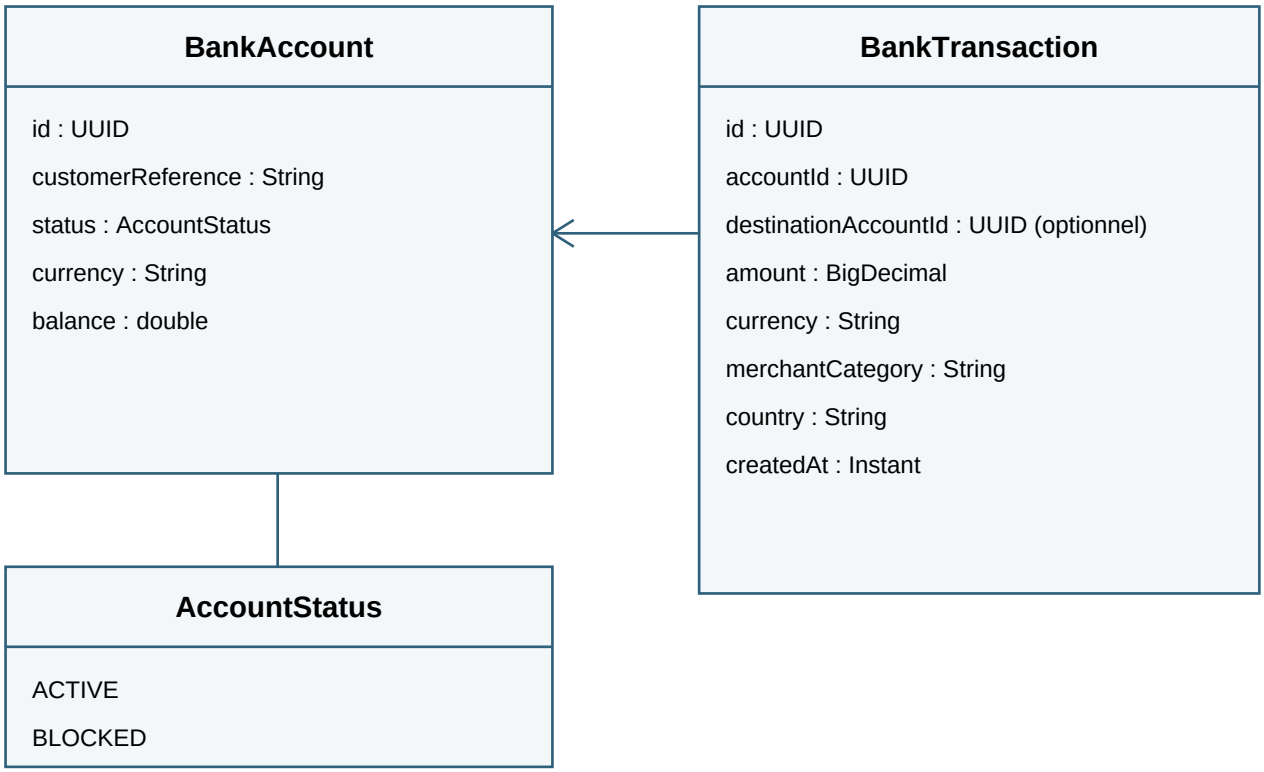


Entité	Attributs principaux
BankAccount	UUID id ; customerReference ; status ACTIVE/BLOCKED ; currency ; double balance
BankTransaction	UUID id ; accountId ; destinationAccountId optionnel ; BigDecimal amount ; currency ; merchantCategory ; country ; Instant createdAt

Les références interservices sont des UUID, sans relation JPA traversant les bases. destinationAccountId complète le modèle minimal pour identifier les virements. Les calculs intermédiaires du solde utilisent BigDecimal ; la précision limitée du double reste documentée.

B - Diagramme de classes métier

Les deux services stockent leurs propres entités. Une transaction référence les comptes par UUID. Le diagramme complet des contrôleurs, outils, services et repositories est fourni dans docs/conception.md.



La flèche entre Transaction et Account exprime une référence métier, pas une clé étrangère ou une jointure entre les bases. destinationAccountId est renseigné pour un virement. Le solde reste un double conformément au modèle minimal fourni, avec des contrôles de précision à la mise à jour.

C - Services métier et connecteurs

Question de l'énoncé : C1 : Account-Service ; C2 : Transaction-Service ; C3 : OpenFeign et Resilience4J.

Pour C1, j'ai construit l'entité, le repository JPA et le service de gestion des comptes, puis les trois connecteurs REST, GraphQL et MCP. Pour C2, j'ai placé les contrôles métier dans le service de transactions afin de les appliquer quel que soit le connecteur. Pour C3, j'ai défini les appels distants vers account-service avec Feign et protégé ces appels avec Resilience4J. Le scénario de débit de 250 EUR vérifie le passage du solde de 1500 à 1250 EUR.

REST, GraphQL et MCP délèguent aux mêmes méthodes de service. La validation ne dépend donc pas seulement du contrôleur REST. Les créations et les virements appliquent les contrôles de montant positif, plafond configuré, existence et activité des comptes, devise compatible et solde suffisant.

La transaction consulte le compte via Feign, persiste son enregistrement, puis demande le débit. Un échec de l'appel entraîne le rollback local. Dans account-service, un virement verrouille les deux comptes dans un ordre stable et réalise le débit et le crédit dans une seule transaction locale.

Connecteur	Comptes	Transactions
REST	/api/accounts ; /{id}/balance	/api/transactions ; /transfers
GraphQL	/graphql ; GraphiQL /graphiql	/graphql ; GraphiQL /graphiql
MCP	/mcp ; 3 outils	/mcp ; 4 outils

Les montants d'entrée GraphQL sont des chaînes décimales, converties en BigDecimal. Les outils MCP sont listAccounts, getAccount, getAccountBalance, listAccountTransactions, getTransaction, createTransaction et transferMoney.

Resilience4J répète uniquement les lectures en échec technique transitoire, jusqu'à trois tentatives. Le circuit ouvre à partir de 50 % d'échecs, après au moins trois appels logiques dans une fenêtre de six. Après dix secondes, deux appels d'essai sont permis. Les erreurs métier 4xx ne sont pas traitées comme des pannes.

Le fallback renvoie HTTP 503 et operation not confirmed. Il ne fabrique ni solde ni succès. Les écritures ne sont pas répétées automatiquement. Le mécanisme ne garantit pas une transaction distribuée entre les bases : une réponse d'écriture perdue peut demander une réconciliation.

Captures reçues : comptes actifs, solde initial 1500 EUR, transaction 250 EUR, puis solde 1250 EUR. Les virements et le fallback disposent également de preuves de tests automatisés, mais aucune capture manuelle supplémentaire n'est inventée.

D - Composants techniques

Question de l’énoncé : D1 : Gateway ; D2 : Eureka ; D3 : Config Server avec un plafond de 1000.

J’ai configuré Gateway comme point d’entrée et défini des routes par nom de service. Eureka fournit le registre de ces noms. J’ai ensuite placé le plafond dans Config Server et configuré son import dans les services. Le dashboard montre les instances UP et l’URL de configuration expose la valeur attendue.

Eureka enregistre les services et permet à OpenFeign et à Gateway de résoudre leurs noms. Config Server utilise le backend native et partage le plafond 1000 depuis les ressources classpath:/config-repo. Les services importent cette configuration au démarrage.

Composant	Port habituel	Responsabilité
Discovery	8761	Registre et dashboard Eureka
Config Server	8888	/transaction-service/default et configuration partagée
Gateway	8080	Routage vers lb://account-service, lb://transaction-service et lb://ai-service
Agent	8083	REST de conversation et liste des outils MCP

Gateway expose /api/accounts, /api/transactions et /api/assistant. Les chemins /accounts/graphql et /transactions/graphql sont réécrits vers /graphql. Les chemins /accounts/mcp et /transactions/mcp sont réécrits vers /mcp.

Les probes Actuator readiness et liveness sont activées. La configuration est chargée au démarrage ; aucun refresh dynamique n’est annoncé. Les tests HTTP finaux utilisent des ports temporaires 18xxx pour éviter des conflits avec les services déjà ouverts.

E - Agent Gemini avec outils MCP

Question de l’énoncé : E1 : Agent utilisant les tools des serveurs MCP ; E2 : test avec un client REST.

J’ai connecté le client MCP de l’agent aux deux serveurs métier et fourni les outils découverts au ChatClient Spring AI. Le LLM peut ainsi consulter les données bancaires par appel d’outil. J’ai exposé une route REST de conversation et utilisé une question sur le solde pour vérifier la chaîne complète : requête REST, appel du modèle, outil MCP, résultat bancaire et réponse.

L’agent utilise le starter Google GenAI de Spring AI et le client MCP WebFlux. Les outils des deux microservices sont découverts à distance puis transmis au ChatClient. La clé est lue via spring.ai.google.genai.api-key=${GEMINI_API_KEY:}, à partir du .env privé.

La consigne système demande de consulter les outils, de ne pas inventer de données et de demander les informations manquantes. Les contenus des outils sont traités comme des données. Une écriture ne doit être exécutée que pour une demande explicite et ne doit pas être répétée automatiquement après un échec.

GET /api/assistant/tools montre les définitions réellement découvertes. POST /api/assistant/chat accepte un message et renvoie une réponse. Un message vide ou mal formé produit HTTP 400. Un modèle désactivé ou un appel défaillant produit HTTP 503.

Question réellement envoyée : Utilise un outil pour consulter le solde du compte 175968ad-f112-416c-9c47-aef217232eb4. Réponds uniquement avec le montant du solde et sa devise.

Résultat de la découverte MCP : PASS. Résultat de génération Gemini : PASS.

Réponse HTTP observée : 200

{"answer": "1250.0 EUR"}

Le test réel valide la découverte des outils, leur appel par Gemini et une réponse cohérente avec le solde du compte. Les quatre tests automatisés de ai-service vérifient également la validation REST et les erreurs explicites.

F - Communication asynchrone Kafka

Question de l'énoncé : F1 : publier après persistance ; F2 : clé accountId ; F3 : consumer ; F4 : journaliser sans modifier les données.

J'ai émis un événement applicatif lors de la création d'une transaction puis utilisé un listener après commit pour l'envoyer à Kafka. Le producer sérialise le contrat JSON demandé et utilise accountId comme clé. Le consumer reçoit le message et écrit uniquement un log. La validation vérifie le contenu reçu et l'absence de nouvelle modification bancaire.

Une transaction réussie émet TransactionCreated dans la couche service. Un listener AFTER_COMMIT publie après le commit JPA sur bank.transactions.created, avec accountId comme clé. Le consumer de transaction-service journalise le topic, la clé et le payload ; il ne modifie aucune donnée bancaire.

Format minimal : eventId (UUID), eventType=TransactionCreated, transactionId (UUID), accountId (UUID), amount (nombre décimal), currency et occurredAt (date ISO-8601). Le topic local comporte trois partitions et une réplication de un. Le partitionnement par compte conserve l'affectation des événements d'un même compte à une partition.

Le test Kafka embarqué vérifie les sept champs, la clé, la réception et l'absence de modification supplémentaire en base. Un second test avec Kafka lancé dans Docker par le candidat a validé un débit MCP de 250 EUR, le passage de 1500 à 1250 EUR et la réception par le consumer.

Événement réellement reçu :

{"eventId":"fc9023ac-8a6c-475f-9333-ac892108d5bf","eventType":"TransactionCreated","transactionId":"b646aa38-c66c-4cc0-b2eb-cea743e3374b","accountId":"ca85c8ff-f174-4f76-b6a0-425f12191038","amount":250,"currency":"EUR","occurredAt":"2026-10-30T11:40:07.110704900Z"}

Limite : AFTER_COMMIT ne rend pas atomiques la base et Kafka. Un arrêt entre le commit et l'envoi ou une indisponibilité prolongée du broker peut faire manquer l'événement. Les erreurs sont journalisées ; aucune outbox persistante n'est annoncée. L'idempotence du producer ne constitue pas une garantie exactly-once de bout en bout.

H - Détection de fraude : réalisation

Le script ml/fraud.py charge bank_transactions.csv et réalise une classification binaire de is_fraud. Les identifiants transaction_id et account_id sont exclus pour éviter de leur attribuer une valeur prédictive artificielle.

La séparation stratifiée conserve la proportion des classes : 4000 lignes d'entraînement et 1000 de test, avec random_state=42. Le jeu complet contient 5000 transactions et 5,14 % de fraudes. Les valeurs manquantes concernent country (66), merchant_category (84) et average_amount_last_7d (75).

Le pipeline utilise une imputation médiane et une standardisation des variables numériques, puis une imputation par la modalité fréquente et un encodage OneHot des catégories. Tous ces traitements sont ajustés uniquement sur l'entraînement. Une régression logistique avec class_weight=balanced est entraînée. Le seuil 0,5 est fixé avant l'évaluation ; il n'est pas choisi sur le test.

Mesure sur le test	Résultat
Précision fraude	0.1221
Rappel fraude	0.6275
F1 fraude	0.2045
Average precision	0.1886

Matrice de confusion - résultat réel

Classe réelle / prédiction	Non-fraude	Fraude
Non-fraude	719	230
Fraude	19	32

Figure produite à partir de ml/results.json, issu de l'exécution du script sur le CSV fourni. Le modèle retrouve 32 fraudes sur 51 mais produit 230 alertes sur des transactions légitimes. Sa précision de 12,21 % impose une validation humaine des alertes ; ces résultats ne justifient pas un blocage automatique des clients.

H - Réponses aux quatre questions

1. Pourquoi l’accuracy seule est-elle insuffisante ? Avec 5,14 % de fraudes, prédire toujours non-fraude donne environ 94,86 % d’accuracy tout en manquant toutes les fraudes. Le rappel, la précision, le F1 et l’average precision évaluent mieux la classe minoritaire.

2. Quel compromis entre faux positifs et fraudes non détectées ? Augmenter les alertes peut réduire les fraudes manquées, mais provoque davantage de contrôles ou de blocages de clients légitimes. Le choix dépend du coût financier des fraudes, du coût des vérifications et de l’impact sur les clients. Ici, 19 fraudes sont manquées et 230 transactions légitimes sont signalées.

3. Effet d’une diminution du seuil ? À scores identiques, davantage de transactions sont classées comme fraudes : le rappel ne diminue pas et les faux positifs peuvent augmenter. La précision baisse souvent, sans garantie de monotonie. Le seuil doit être choisi avec un jeu de validation ou une validation croisée sur l’entraînement, jamais avec le test final.

4. Pourquoi une validation temporelle ? Elle entraîne sur le passé et évalue sur le futur, ce qui reflète le déploiement et l’évolution des fraudes. Elle évite les fuites du futur. Les variables historiques doivent être calculées avec les seules observations antérieures. Le CSV fourni contient une heure mais pas une date complète : le travail applique la séparation stratifiée demandée ; hour ne permet pas de reconstruire une chronologie fiable.

Reproduction : python ml/fraud.py chemin/vers/bank_transactions.csv --output ml/results.json. Dépendances : ml/requirements.txt. Le modèle reste expérimental ; son intégration aux microservices n’est pas demandée.

I - Docker et intégration continue

I1. Un Dockerfile par microservice utilise une image Java 17, copie le JAR compilé et lance java -jar sous un utilisateur non privilégié. Les JAR sont produits par mvn -B verify avant la construction des images. Le .dockerignore exclut les secrets et les sources inutiles au contexte de construction.

I2. docker-compose.yml décrit les six services Java et Kafka. Les services communiquent par leurs noms Docker ; le broker annonce kafka:9092. Les URL Config Server, Eureka et MCP sont passées par variables d'environnement. Kafka possède un healthcheck. depends_on ordonne les démarrages, sans garantir la disponibilité des services Java ; restart:on-failure permet leur relance en cas d'échec initial.

I3. docs/github-actions-example.yml compile et teste avec Maven, construit les images et les publie sur GHCR avec un tag égal au SHA Git. Une étape de déploiement manuel utilise un runner bank-demo préconfiguré et docker compose pull puis up -d --no-build. Cet extrait nécessite ce runner, les droits de registre et le secret Gemini. Aucun déploiement cloud ni lancement du pipeline n'est revendiqué.

Livrable	État
6 Dockerfiles	Fournis
docker-compose.yml complet	Fourni ; lancement complet non exécuté
Workflow GitHub Actions	Fourni ; pipeline non exécuté
Kafka Docker	Exécuté et vérifié précédemment
Bases H2	En mémoire ; données perdues au redémarrage

Commandes locales : mvn -B verify ; docker compose config --quiet ; docker compose up -d --build. Les ports habituels doivent être libres. Le fichier docs/devops.md précise les prérequis et les limites.

I - Réponse courte DevOps

Secrets et configuration : les URL, ports et plafonds appartiennent à la configuration. Les clés API et mots de passe sont des secrets injectés par variables d'environnement ou par un gestionnaire de secrets. Le .env reste local et ignoré par Git ; la CI utilise les secrets GitHub. Aucun secret ne doit être intégré aux images ni imprimé dans les logs.

Démarrage, disponibilité, vitalité : un contrôle de démarrage laisse le temps à Java de démarrer ; readiness indique si le service peut recevoir des requêtes ; liveness indique si le processus fonctionne. Actuator expose les groupes /actuator/health/readiness et /actuator/health/liveness. Ces contrôles doivent avoir des délais adaptés. Le compose fourni n'implémente pas encore des probes HTTP pour chaque service Java.

Logs, métriques, traces : centraliser les logs structurés facilite la recherche d'un incident entre services. Les métriques suivent taux d'erreurs, latence et saturation. Les traces distribuées relient les appels Gateway, Feign et services au moyen d'identifiants de trace. La réalisation utilise les logs applicatifs ; aucune infrastructure complète de collecte de métriques ou de traces n'a été déployée.

Rollback : conserver les images identifiées par SHA, remettre IMAGE_TAG au tag précédent, puis exécuter docker compose pull et docker compose up -d --no-build. Vérifier ensuite la disponibilité et les tests REST. Le rollback applicatif ne restaure pas les données : les changements de schéma doivent rester compatibles et les sauvegardes sont traitées séparément.

Validation et limites du livrable

Le build validé compile le parent et les six modules avec 31 tests réussis. Le test Gemini connecté aux outils MCP a également réussi. Les tests HTTP finaux et les validations antérieures sont conservés dans docs/verification-*.json et docs/verification-*.txt. Les preuves mentionnent le contexte du test, notamment broker embarqué, broker Docker ou doubles de test.

Les captures ci-après proviennent du candidat. Celles des outils MCP montrent les réponses réellement enregistrées de tools/list, affichées ensuite dans le navigateur ; ce ne sont pas des nouveaux endpoints métier. Les captures de l’agent, de GraphiQL et de Gateway restent à ajouter si elles n’ont pas encore été transmises.

La partie G reste non réalisée. Un compose complet et un pipeline CI sont fournis dans la partie I ; leur déploiement complet n’a pas été exécuté. Aucune soumission Classroom ni publication cloud n’est déclarée réalisée.

Sources : énoncé Test ADRIA ENSET 2027.pdf fourni par le candidat ; code et résultats du dépôt ; documentation officielle Spring AI (Google GenAI et MCP), Spring Cloud et Spring GraphQL. Les divergences volontaires et limites de cohérence sont explicitées dans les sections correspondantes.

Résultat du test REST de l’agent

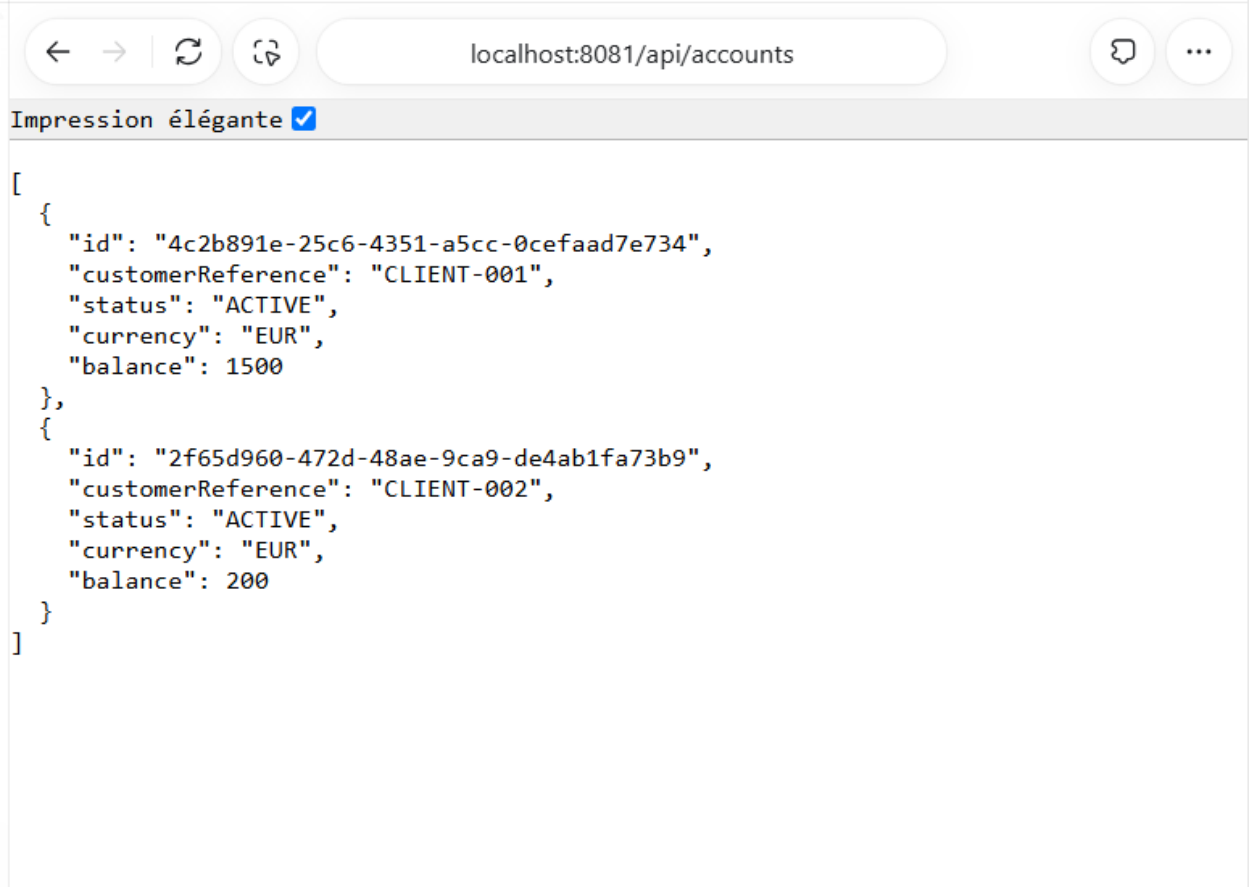
Présentation du résultat enregistré dans docs/demo-agent.json lors du test réel. Cet encadré est une restitution du résultat, et non une capture du navigateur.

Élément	Valeur observée
Requête	Utilise un outil pour consulter le solde du compte 175968ad-f112-416c-9c47-aef217232eb4. Réponds uniquement avec le montant du solde et sa devise.
Code HTTP	200
Réponse	1250.0 EUR

La réponse correspond au solde après débit : 1500 - 250 = 1250 EUR. La découverte MCP et la génération Gemini ont réussi lors de ce test.

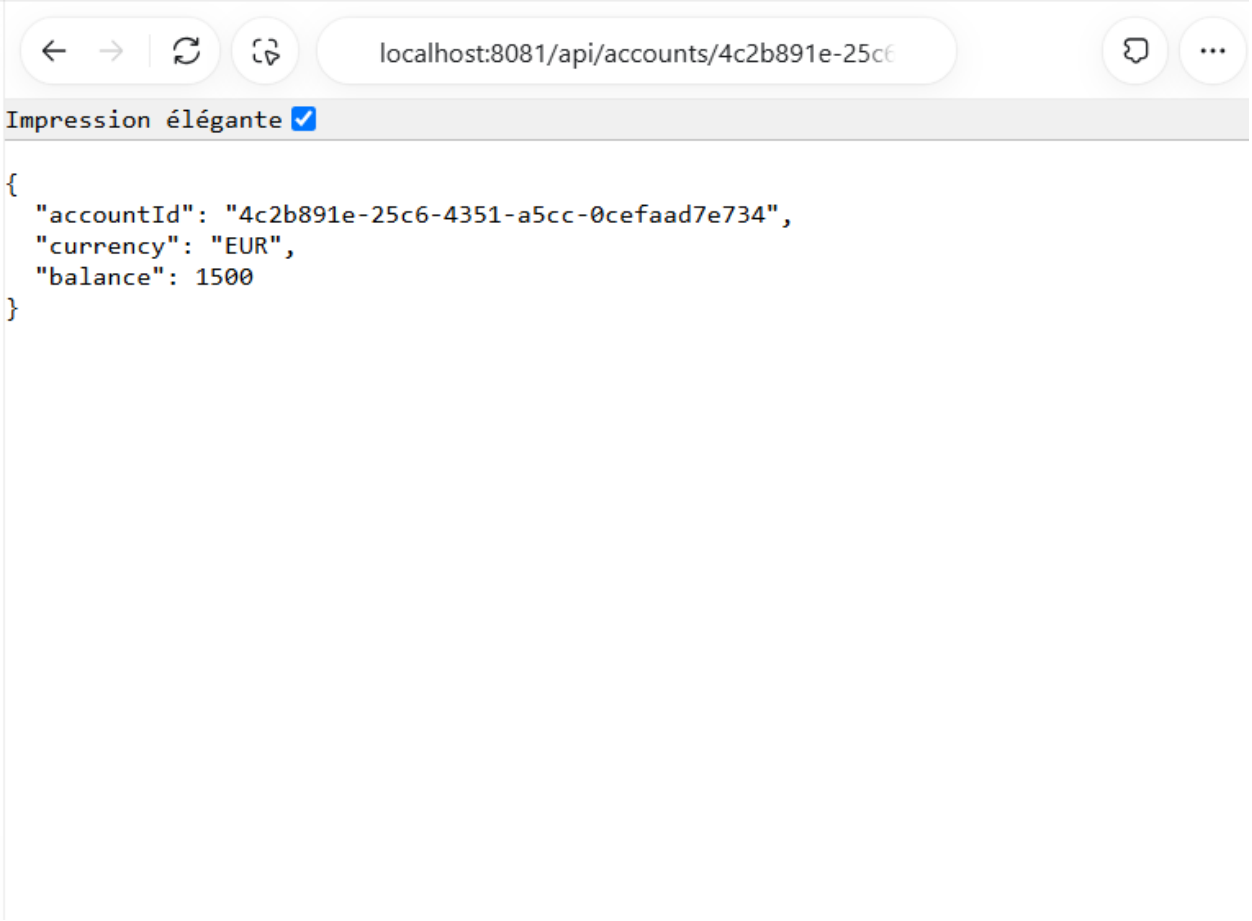
Capture - 02-01-comptes.png

Comptes actifs et soldes initiaux : 1500 EUR et 200 EUR.



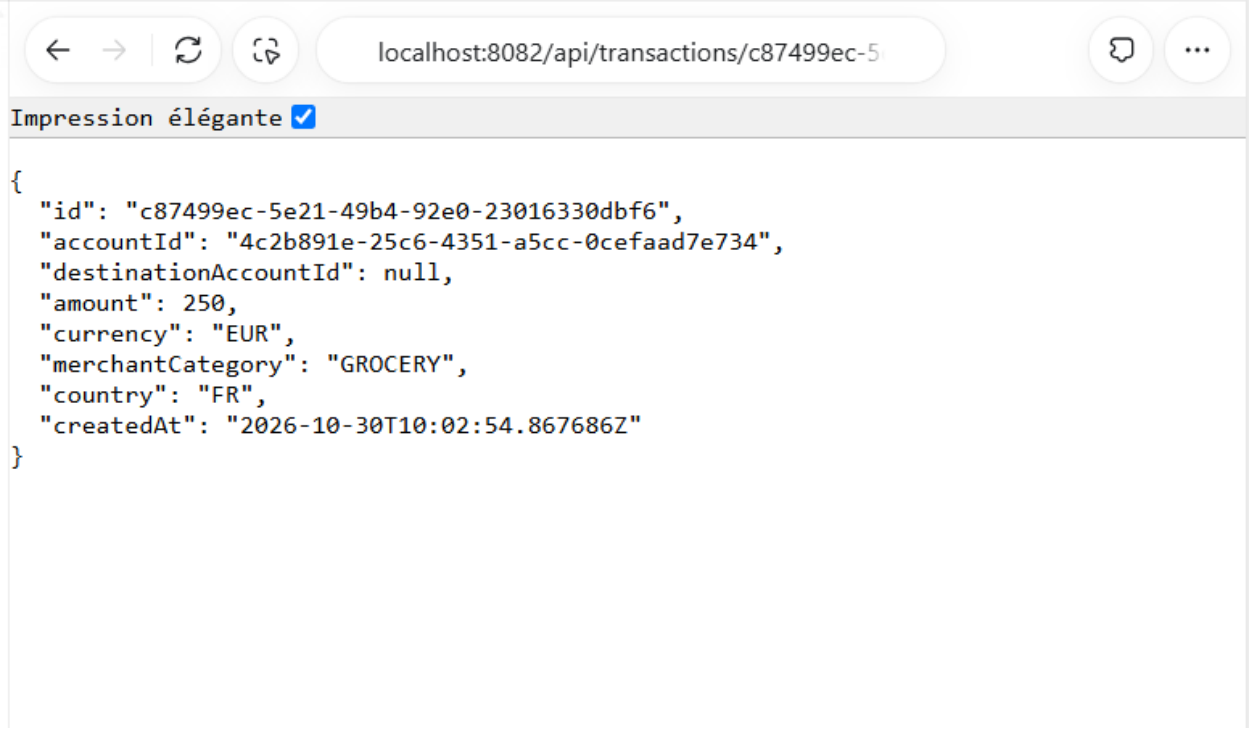
Capture - 02-02-solde-avant.png

Solde du compte source avant débit : 1500 EUR.



Capture - 02-03-transaction.png

Lecture de la transaction créée de 250 EUR ; ce GET ne montre pas le statut 201 du POST.



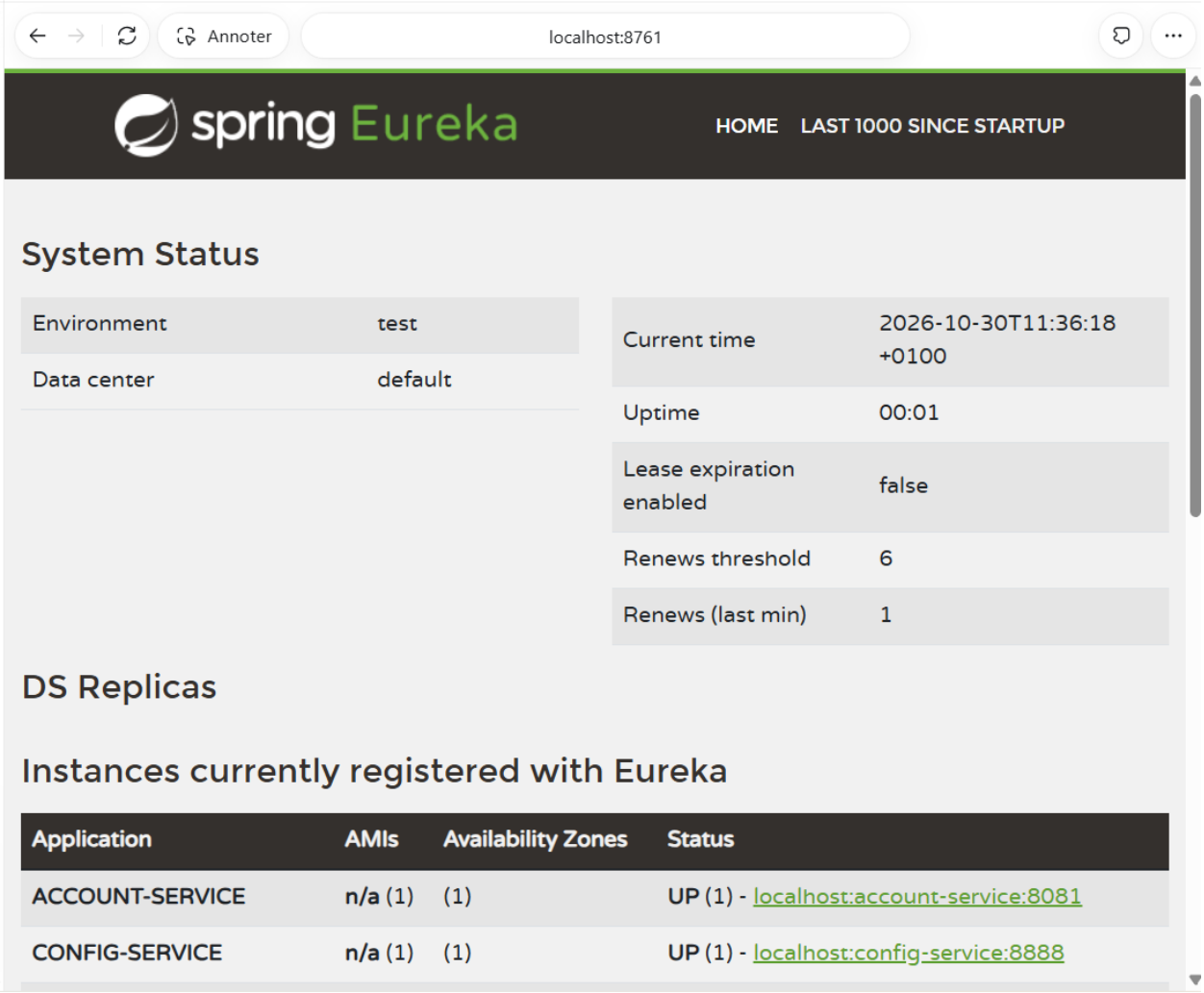
Capture - 02-04-solde-apres.png

Solde après le débit : 1250 EUR.



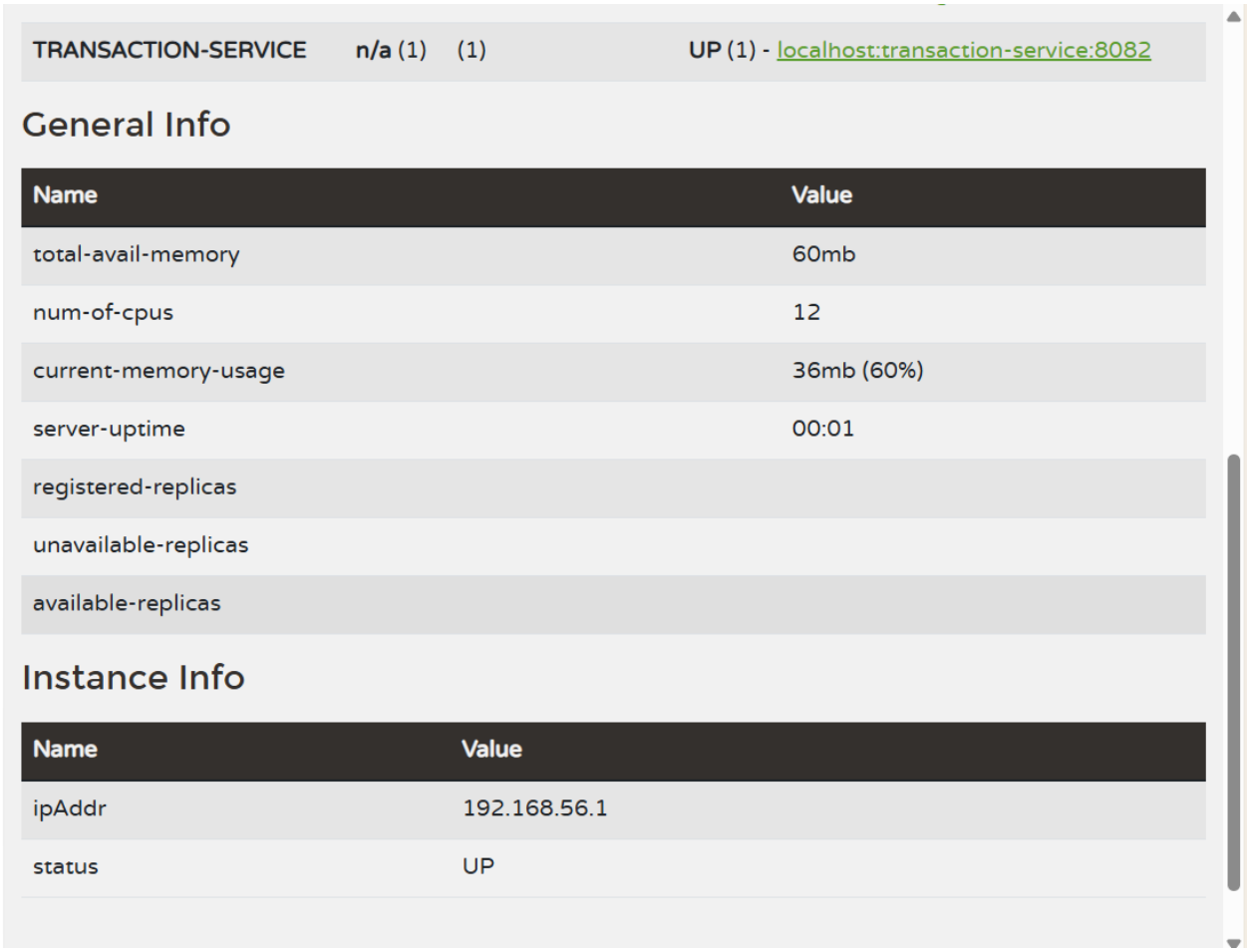
Capture - 03-01-eureka-a.png

Dashboard Eureka : account-service et config-service enregistrés et UP.



Capture - 03-01-eureka-b.png

Suite du dashboard : transaction-service également UP.



TRANSACTION-SERVICE	n/a (1)	(1)	UP (1) - localhost:transaction-service:8082

General Info

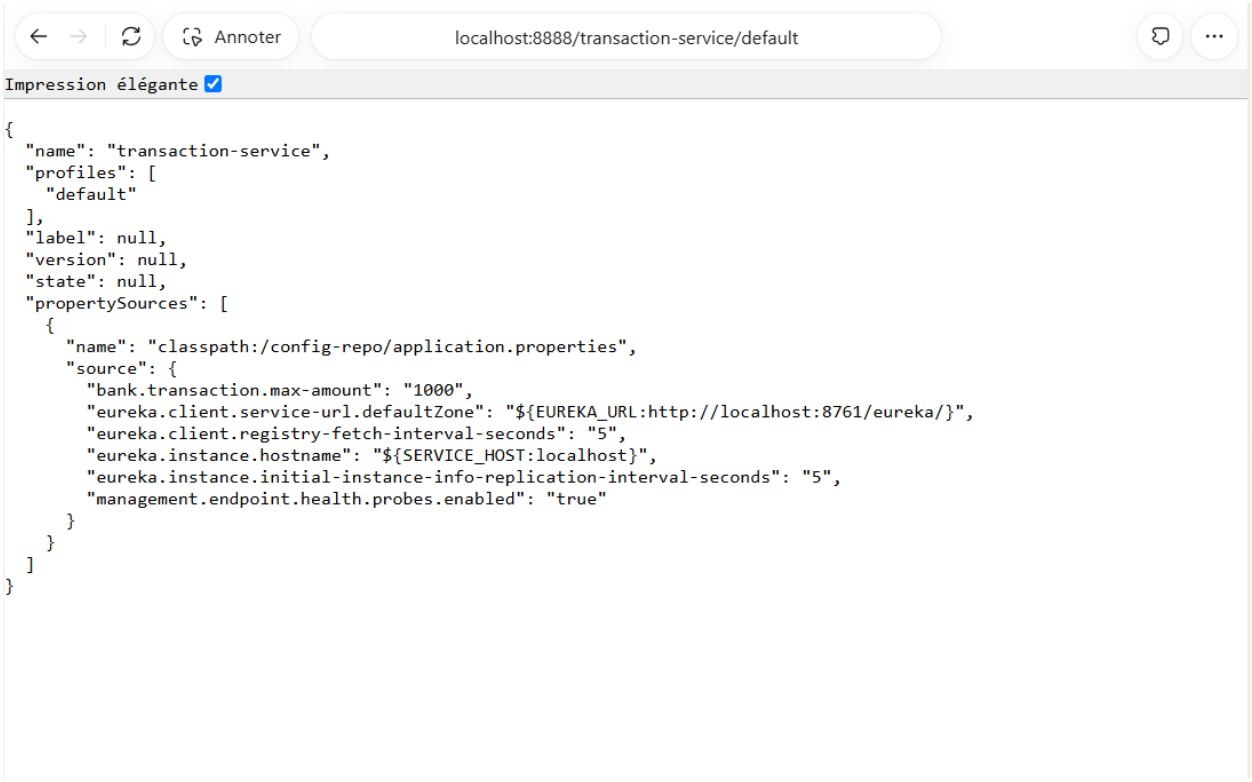
Name	Value
total-avail-memory	60mb
num-of-cpus	12
current-memory-usage	36mb (60%)
server-uptime	00:01
registered-replicas	
unavailable-replicas	
available-replicas	

Instance Info

Name	Value
ipAddr	192.168.56.1
status	UP

Capture - 03-02-config.png

Config Server : bank.transaction.max-amount=1000.



Capture - 04-04-account-tools-a.png

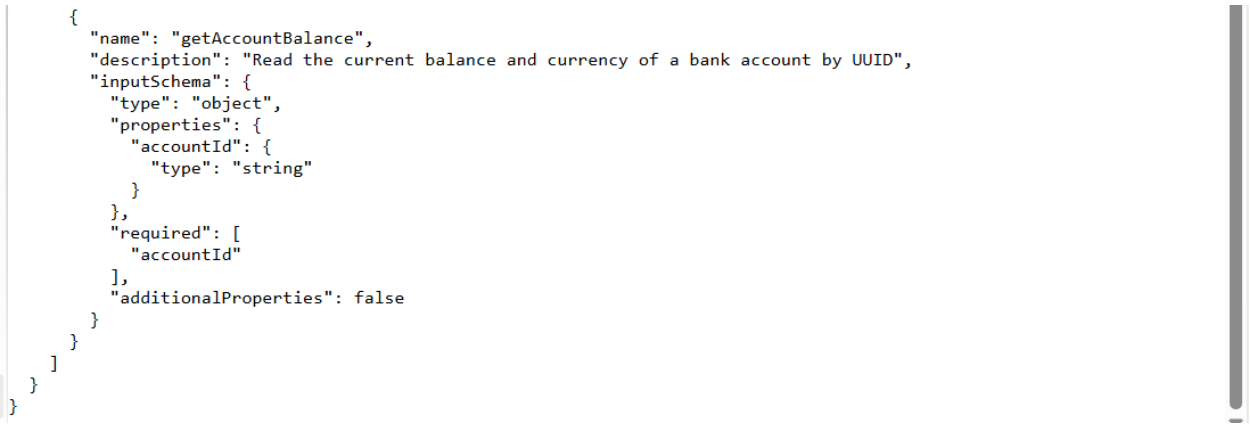
Réponse MCP enregistrée tools/list : outils du service des comptes.


```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "tools": [
      {
        "name": "listAccounts",
        "description": "List all bank accounts and their identifiers",
        "inputSchema": {
          "type": "object",
          "properties": {

          },
          "required": [],
          "additionalProperties": false
        }
      },
      {
        "name": "getAccount",
        "description": "Get an existing account, its status, currency and balance by UUID",
        "inputSchema": {
          "type": "object",
          "properties": {
            "accountId": {
              "type": "string"
            }
          },
          "required": [
            "accountId"
          ],
          "additionalProperties": false
        }
      },
      {
        "name": "getAccountBalance",
        "description": "Read the current balance and currency of a bank account by UUID",
        "inputSchema": {
          "type": "object",
          "properties": {
            "accountId": {
              "type": "string"
            }
```

Capture - 04-04-account-tools-b.png

Suite du schéma de getAccountBalance ; accountId est requis.



```
      {
        "name": "getAccountBalance",
        "description": "Read the current balance and currency of a bank account by UUID",
        "inputSchema": {
          "type": "object",
          "properties": {
            "accountId": {
              "type": "string"
            }
          },
          "required": [
            "accountId"
          ],
          "additionalProperties": false
        }
      }
    ]
  }
}
```